Symmetry Acquisition in Predictive Coding Networks

Adam Shaw [1] **Jiayu Li** [1] **Michael Sperling** [1] **Michael Kim** [1] **Alvin Jin** [1]

Abstract

Predictive coding networks (PCNs) perform inference through recurrent, bidirectional dynamics, requiring representations that support both abstraction and approximate inversion. We study how PCNs learn representations through the lens of weight-space symmetry, using persistent homology to track when and where PCN layers merge previously distinct regions of the data manifold. This provides a topological probe of representation-space identifications induced by the network's learned dynamics. We find that this merging occurs across layers, but its timing is strongly controlled by model capacity ($0.72 \leq \rho \leq 0.79$) and activation smoothness. Critically, networks that merge representations earlier perform worse in input reconstruction tasks ($\rho = -0.58$). This reveals a measurable cost of early representational symmetry acquisition in generative architectures, and demonstrates that topological measures can expose structural properties of learned representations that loss values alone cannot.

1. Introduction

A growing body of work connects representational structure in neural networks to weight-space symmetries, with particular interest in how these symmetries shape learned representations. One concrete manifestation is topological simplification: prior work shows that feedforward networks progressively reduce the Betti numbers of data manifolds, acquiring *distinguishability symmetries* that identify previously distinct points in representation space (Naitzat et al., 2020; Ergen & Grillo, 2024; Suresh et al., 2024). For discriminative architectures, this symmetry acquisition is benign. For generative architectures, however, it carries a direct cost: once homologically distinct regions are identified, the information required for faithful reconstruction is irreversibly lost.

We study this cost in predictive coding networks (PCNs) (Rao & Ballard, 1999; Whittington & Bogacz, 2017), recurrent bidirectional models that perform inference by minimizing local prediction error and support approximate inversion (Sun & Orchard, 2020). PCNs constitute a theoretically motivated setting for studying the tension between representational symmetry and invertibility. Unlike feedforward networks, they must simultaneously abstract and reconstruct, making the trade-off between symmetry acquisition and inversion fidelity directly measurable.

Using persistent homology as an architecture-agnostic probe, we show that capacity and activation smoothness govern when PCNs impose representational identifications across layers, and that earlier identification is associated with significantly worse reconstruction. These results provide representation-level evidence that topological measures can expose hidden symmetry structure in generative architectures that is invisible to standard loss-based diagnostics.

2. Background

2.1. Betti Numbers and Persistent Homology

Betti numbers β_k count k-dimensional holes in a topological space. Persistent homology estimates these from finite point clouds by tracking how features appear and disappear across a filtration parameter (Edelsbrunner et al., 2002), with long-lived features reflecting meaningful structure. Prior work shows that deep networks progressively reduce Betti numbers across layers (Naitzat et al., 2020; Suresh et al., 2024; Ergen & Grillo, 2024). In our setting, each such reduction marks a layer at which the network commits to a new representational identification, making the *timing* of Betti collapse the central quantity of interest.

2.2. Predictive Coding Networks

PCNs are hierarchical generative models that perform inference by minimizing a global energy functional via recurrent, bidirectional dynamics (Rao & Ballard, 1999; Whittington & Bogacz, 2017; Stenlund, 2025). Unlike feedforward networks, PCNs support approximate inversion by clamping

[1]Department of Mathematics, University of Southern California, Los Angeles, CA, United States. Correspondence to: Alvin Jin <adshaw@usc.edu>.

Workshop on Weight-Space Symmetries, held in conjunction with the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. 2026.

higher-level states and inferring lower-level representations through the same dynamics (Sun & Orchard, 2020). This makes topological simplification directly costly: each identification imposed by the encoder shrinks the space of inputs recoverable by inversion, motivating our use of topological measures to study the abstraction–invertibility trade-off. We lay out more details regarding our PCN formulation in Appendix A.

3. Methodology

We evaluate how architectural choices affect the timing of topological simplification in PCNs. Specifically, we vary model capacity and activation function, compute persistent homology on layerwise representations, and analyze when the network commits to identifying previously distinct regions of the data manifold. A complete list of all architectures is provided in Appendix B.

Throughout this work, we refer to a model as having L hidden layers, with layer indices $\ell \in \{0, \dots, L+1\}$ including the input and output layers.

3.1. Datasets

We evaluate on (i) a synthetic dataset with known topology, adapted from Naitzat et al. (2020), consisting of nine disjoint disks embedded in a single connected region with nine holes, enabling controlled analysis of ground-truth symmetry structure, and (ii) MNIST, to assess whether observed trends generalize to real-world data (LeCun et al., 1998).

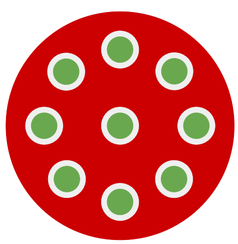


Figure 1. Visualization of the synthetic dataset, consisting of manifold M_a (green) embedded in manifold M_b (red). Dataset and figure both adapted from Naitzat et al. (2020).

3.2. Topology Tracking

For each trained model, we extract layerwise activations for a fixed dataset and compute persistent homology on the resulting point clouds. We use normalized distance metrics (k-NN geodesic for synthetic data, normalized Euclidean for MNIST) to mitigate arbitrary scaling differences across layers and ensure comparability. From persistence, we compute Betti numbers (primarily β_0 and β_1) at a fixed filtration scale, yielding a scalar summary of topological structure per layer. Our full topology tracking procedure is laid out in Appendix C.

3.3. Metrics

We define two new metrics: one to quantify timing of topological simplification, and another to quantify reconstruction fidelity.

Let $B(\ell) = \sum_{k \in \mathcal{K}} w_k \beta_k^{(\ell)}$ be a weighted sum of Betti numbers at layer ℓ, for some range of homological degrees (e.g., $\mathcal{K} = \{0, 1\}$), and let $\tilde{B}(\ell) = \min_{j \leq \ell} B(j)$ be its running minimum, so that simplification events are treated as irreversible. The layer-wise Betti drop $\Delta(\ell) = \tilde{B}(\ell-1) - \tilde{B}(\ell)$ then defines a distribution over layers $p(\ell) = \Delta(\ell)/D$, where $D = \sum_\ell \Delta(\ell)$ is the total simplification across all layers.

Definition 3.1 (COM)**.** To quantify when simplification occurs, we define the *Center of Mass of Topological Simplification* (COM) as the expected depth of topological simplification under this distribution:

$$\mathrm{COM} = \sum_{\ell=1}^{L+1} \ell \, p(\ell) = \frac{\sum_{\ell=1}^{L+1} \ell \, \Delta(\ell)}{\sum_{\ell=1}^{L+1} \Delta(\ell)}. \tag{1}$$

A lower COM indicates earlier topological collapse. For conciseness, we write $\mathrm{COM}(\beta_0 + \beta_1)$ as shorthand to indicate that $B(\ell) = \beta_0^{(\ell)} + \beta_1^{(\ell)}$, and similarly for other linear combinations of Betti numbers.

Definition 3.2 (MRD)**.** To quantify reconstruction fidelity, we measure how closely inverted outputs resemble valid same-class inputs. For each of $M = 30$ independently-trained models m of an architecture A, we pass $N = 1000$ synthetic one-hot outputs y_i through the inversion procedure (see Appendix A.2), each yielding a reconstruction $\hat{x}_i^{(m)}$. We then find its nearest Euclidean neighbor $x_i^{*(m)}$ in the training set restricted to the same class.

We define the *Mean Reconstruction Distance* (MRD) of an architecture A as the mean Euclidean distance between reconstructions and their nearest same-class neighbors, averaged over N reconstructions and M model seeds:

$$\mathrm{MRD}(A) = \frac{1}{MN} \sum_{m=1}^{M} \sum_{i=1}^{N} \left\| \hat{x}_i^{(m)} - x_i^{*(m)} \right\|_2. \tag{2}$$

By comparing to the nearest same-class neighbor rather than a fixed target, MRD captures whether inversions lie near the correct data manifold without penalizing valid novel reconstructions.

Together, COM and MRD form a theoretically unified pair: COM localizes the layer at which the network imposes topological simplification, and MRD quantifies the downstream cost of that symmetry for the generative task. The strong negative correlation we find between the two indicates that earlier symmetry acquisition is associated with degraded invertibility.

4. Results and Discussion

Topological simplification corresponds to the progressive acquisition of approximate symmetries in representation space. Each decrease in Betti numbers reflects the identification of previously distinguishable regions of the input manifold under the learned mapping $f_\theta : \mathcal{X} \to \mathcal{Z}$. This induces an approximate equivalence relation

$$x \sim x' \quad \Longleftrightarrow \quad f_\theta(x) \approx f_\theta(x'),$$

which partitions the input space into equivalence classes. The primary quantity of interest is therefore the depth at which these identifications occur.

4.1. Capacity Controls the Timing of Symmetry Acquisition

Model capacity determines the depth at which symmetries are imposed. Larger models consistently delay symmetry acquisition to deeper layers, while smaller models impose symmetries earlier in the hierarchy. This relationship is robust across activation functions, with strong positive correlations between model size (measured as the sum of hidden-layer widths) and COM in the range of 0.72–0.79 (see Figure 2).

Activation-dependent capacity sensitivity. To compare the strength of this capacity effect across activation functions, we fit a separate linear model within each activation family,

$$\mathrm{COM}_a(P) = \alpha_a + \gamma_a P + \varepsilon,$$

where $P = \sum_{\ell=1}^{L} n_\ell$ is the total hidden width of the architecture and a denotes the activation function. The fitted slopes were positive for all three activation families: $100\hat{\gamma} = 1.79$ for ReLU, 0.96 for Leaky ReLU, and 0.91 for $\tanh$, with corresponding R^2 values of 0.59, 0.63, and 0.41, respectively. Thus, while increasing capacity delays simplification for all activations, the effect is strongest for ReLU: its slope is nearly twice that of $\tanh$. This suggests that ReLU PCNs are more capacity-sensitive, in the sense that added width more strongly delays the layer at which topological collapse occurs.

One interpretation is that ReLU networks, which can impose sharper non-homeomorphic contractions, simplify especially early when capacity is limited; additional capacity partially offsets this tendency by allowing representations to preserve topological distinctions deeper into the network.

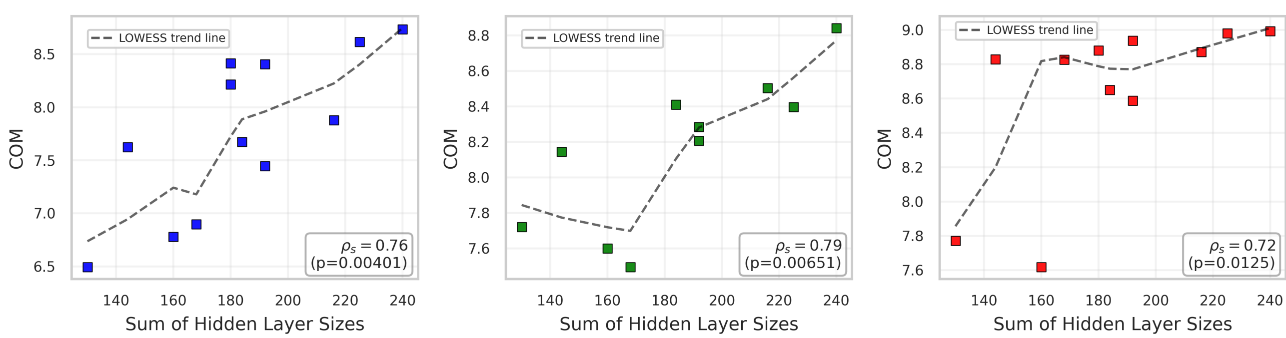
4.2. A Trade-off Between Symmetry and Invertibility

A strong negative correlation $\rho = -0.58$ between COM and reconstruction quality (MRD) shows that earlier symmetry acquisition degrades inversion performance (Figure 3).

Identifying distinct regions of the input manifold induces a quotient structure over $\mathcal{X}$. Once two regions are mapped to the same representation, the information required to distinguish them is irreversibly lost. Subsequent layers cannot recover this information.

This yields the following trade-off:

- Early symmetry acquisition $\Rightarrow$ coarse equivalence classes $\Rightarrow$ strong abstraction, weak invertibility
- Late symmetry acquisition $\Rightarrow$ finer distinctions preserved $\Rightarrow$ weaker abstraction, strong invertibility



(a) ReLU Activations *(b)* Leaky ReLU Activations *(c)* $\tanh$ Activations

Figure 2. Model size versus simplification timing on the synthetic dataset, shown separately for each activation function. Within each activation (a) ReLU, (b) Leaky ReLU, and (c) $\tanh$, larger networks (greater total hidden width) exhibit higher COM, indicating that topological simplification is delayed to later layers; Spearman's correlation and p-values are reported in each panel, with LOWESS fits shown as dashed lines. We stratify by activation function because it is a confounding variable that influences both representation geometry and simplification behavior.

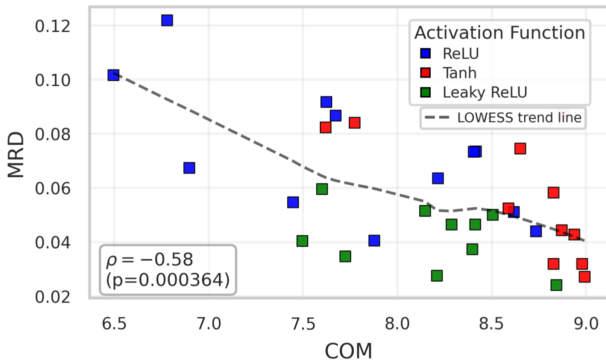


Figure 3. Graphical relationship and correlation between early topological simplification ($\mathrm{COM}(\beta_0)$) and reconstruction error (MRD) in architectures trained on the synthetic dataset. The strong negative correlation highlights a clear association between earlier topological simplification and weaker reconstruction performance.

Invertibility therefore requires delaying symmetry acquisition. Reconstruction quality depends on how long the network preserves distinguishability before imposing invariances.

4.3. Activation Functions Control the Regularity of Symmetry Transformations

Activation smoothness determines how symmetries are introduced. Non-smooth activations (ReLU, Leaky ReLU) induce earlier and more abrupt symmetry acquisition, while smooth activations ($\tanh$) delay it. This is consistent with prior work linking non-smooth nonlinearities to stronger topological contraction, and suggests that activation choice interacts with PCN inference dynamics to shape representational structure (Naitzat et al., 2020; Ergen & Grillo, 2024).

Non-smooth nonlinearities produce hard identifications, collapsing regions abruptly. Smooth nonlinearities produce gradual, approximate identifications that preserve distinctions over a wider range of layers.

These results indicate that the symmetries induced by neural networks are inherently approximate and strongly shaped by architectural choices.

4.4. Data-Dependent Symmetry Structure on MNIST

On MNIST, representations exhibit the same qualitative behavior: progressive symmetry acquisition across layers, with smaller models imposing symmetries earlier.

Capacity effects are weaker for non-smooth activations, suggesting that high-dimensional data and activation-induced distortions obscure the underlying symmetry structure. Despite this, distributional comparisons of COM values (Figure 4) show that the ordering of symmetry acquisition with respect to capacity persists, particularly for smooth activations.

These observations indicate that the symmetry structure of representations extends to real-world data, although it is less directly observable.

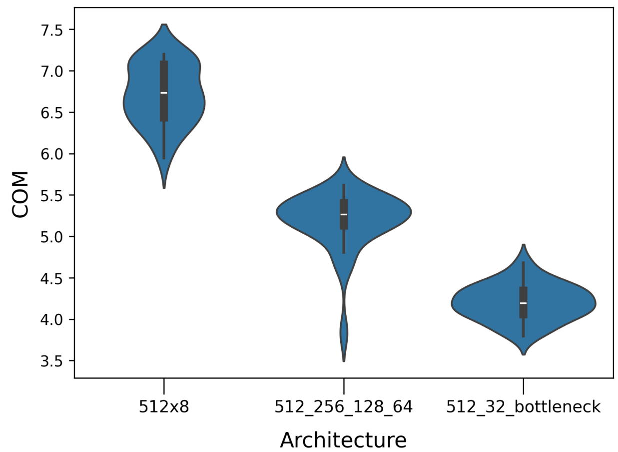


Figure 4. Violin plots of $\mathrm{COM}(\beta_0+\beta_1)$ across $\tanh$ architectures of decreasing hidden-layer sizes trained on MNIST, computed from 30 independently trained networks per configuration. Lower COM values indicate that topological simplification occurs earlier in the network. For meaning of architecture names, refer to Table B.2.

5. Conclusion

In this work, we investigate how representational topology evolves across layers in PCNs and how this evolution depends on model capacity and activation smoothness. We find that smaller models and non-smooth activations acquire representational symmetries earlier, while larger models and smoother nonlinearities delay this process. We interpret these Betti-number reductions as representational identifications: previously distinct regions of the data manifold are merged under the learned inference dynamics.

Crucially, earlier topological simplification is associated with worse reconstruction fidelity, revealing a trade-off between abstraction and invertibility. Symmetry acquisition supports compression, but removes information needed for approximate inversion. These results position persistent homology as a useful probe of symmetry structure in PCNs, exposing representational dynamics that are not visible from loss values alone.

Further work can extend these analyses to larger generative architectures and modern representation-learning systems, as well as investigate how these phenomena evolve across datasets with varying structural and feature complexity.

References

Attali, D., Lieutier, A., and Salinas, D. Vietoris–rips complexes also provide topologically correct reconstructions of sampled shapes. *Computational Geometry*, 46(4): 448–465, May 2013. doi: 10.1016/j.comgeo.2012.02.009.

Bauer, U. Ripser: efficient computation of vietoris-rips persistence barcodes. *Journal of Applied and Computational Topology*, 5(3):391–423, 2021.

Edelsbrunner, H. and Harer, J. L. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

Edelsbrunner, H., Letscher, D., and Zomorodian, A. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, Nov 2002. doi: 10.1007/s00454-002-2885-2.

Ergen, E. and Grillo, M. Topological expressivity of relu neural networks. *Proceedings of Machine Learning Research*, 196:1–44, 2024. URL https://proceedings.mlr.press/v247/ergen24a/ergen24a.pdf.

LeCun, Y., Cortes, C., and Burges, C. J. MNIST handwritten digit database. http://yann.lecun.com/exdb/mnist/, 1998.

Naitzat, G., Zhitnikov, A., and Lim, L. Topology of deep neural networks. *Journal of Machine Learning Research*, 21(184):1–40, 2020. URL http://jmlr.org/papers/v21/20-345.html.

Pinchetti, L., Qi, C., Lokshyn, O., Olivers, G., Emde, C., Tang, M., M'Charrak, A., Frieder, S., Menzat, B., Bogacz, R., Lukasiewicz, T., and Salvatori, T. Benchmarking predictive coding networks – made simple, 2025. URL https://arxiv.org/abs/2407.01163.

Rao, R. P. N. and Ballard, D. H. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2 (1):79–87, 1999. doi: 10.1038/4580.

Stenlund, M. Introduction to Predictive Coding Networks for Machine Learning. *arXiv e-prints*, art. arXiv:2506.06332, May 2025. doi: 10.48550/arXiv.2506.06332.

Sun, W. and Orchard, J. A predictive-coding network that is both discriminative and generative. *Neural Computation*, 32(10):1836–1862, Oct 2020. doi: 10.1162/neco_a_01311.

Suresh, S., Das, B., Abrol, V., and Dutta Roy, S. On characterizing the evolution of embedding space of neural networks using algebraic topology. *Pattern Recognition Letters*, 179:165–171, Mar 2024. doi: 10.1016/j.patrec.2024.02.003.

Wheeler, M., Bouza, J., and Bubenik, P. Activation landscapes as a topological summary of neural network performance. *2021 IEEE International Conference on Big Data (Big Data)*, pp. 3865–3870, Dec 2021. doi: 10.1109/bigdata52589.2021.9671368.

Whittington, J. C. R. and Bogacz, R. An approximation of the error backpropagation algorithm in a predictive coding network. *Neural Computation*, 29(5):1229–1262, 2017. doi: 10.1162/NECO_a_00949.

Zomorodian, A. and Carlsson, G. Computing persistent homology. *Discrete & Computational Geometry*, 33(2): 249–274, Nov 2004. doi: 10.1007/s00454-004-1146-y.

A. Predictive Coding Network Formulation

We implement our models using the JAX-based `PCX` library (Pinchetti et al., 2025), which follows the formulation of PCNs introduced by Whittington & Bogacz (2017).

Each layer ℓ contains neural activities $x_i^{(\ell)}$, where $i \in \{1, \ldots, n_\ell\}$ indexes units within the layer. Top-down predictions are defined via a generative model. The predicted mean activity $\mu_i^{(\ell)}$ at layer ℓ is given by

$$\mu_i^{(\ell)} = \sum_{j=1}^{n_{\ell+1}} \theta_{i,j}^{(\ell+1)} \, f\left(x_j^{(\ell+1)}\right), \tag{A.1}$$

where $\theta_{i,j}^{(\ell+1)}$ denotes the synaptic weight from unit j in layer $\ell+1$ to unit i in layer ℓ, and $f(\cdot)$ is a (possibly nonlinear) activation function.

Prediction errors are defined as variance-normalized residuals

$$\epsilon_i^{(\ell)} = \frac{x_i^{(\ell)} - \mu_i^{(\ell)}}{\Sigma_i^{(\ell)}}, \tag{A.2}$$

where $\Sigma_i^{(\ell)}$ denotes the variance associated with unit i at layer ℓ.

Under a Gaussian noise assumption, the variational free energy is given by

$$F = -\frac{1}{2} \sum_{\ell=0}^{\ell_{\max}-1} \sum_{i=1}^{n_\ell} \frac{\left(x_i^{(\ell)} - \mu_i^{(\ell)}\right)^2}{\Sigma_i^{(\ell)}}. \tag{A.3}$$

Inference corresponds to gradient descent on this free energy with respect to the neural activities. The resulting continuous-time dynamics for unit b in layer a are

$$\dot{x}_b^{(a)} = -\epsilon_b^{(a)} + \sum_{i=1}^{n_{a-1}} \epsilon_i^{(a-1)} \, \theta_{i,b}^{(a)} \, f'\left(x_b^{(a)}\right), \tag{A.4}$$

where $f'(\cdot)$ denotes the derivative of the activation function.

Learning proceeds via gradient descent on the variational free energy with respect to the synaptic parameters. The resulting gradient is given by Whittington & Bogacz (2017, Eq. (2.21)):

$$\frac{\partial F}{\partial \theta_{i,j}^{(\ell)}} = -\epsilon_i^{(\ell-1)} \, f\left(x_j^{(\ell)}\right). \tag{A.5}$$

This confirms that both inference and learning depend only on locally available pre- and post-synaptic signals.

Because inference is defined by minimizing a shared energy functional rather than propagating signals through a fixed forward pathway, PCNs naturally admit approximate inversion by clamping higher-level states and inferring lower-level representations through the same dynamics. Despite growing interest in PCNs across neuroscience and machine learning, the topological consequences of these recurrent inference dynamics remain largely unexplored, motivating the present study.

A.1. Contrast with Backpropagation

PCNs offer a biologically plausible alternative to backpropagation (BP) by addressing the *weight transport problem*: in BP, synaptic updates require error signals to be propagated backward through the exact transposes of downstream weight matrices, a globally coordinated and non-local operation unsupported by known biological mechanisms. In PCNs, however, all computations are strictly local: a neuron only requires information from its immediate neighbors. Remarkably, despite this locality constraint, these update rules approximate backpropagation to a high degree (Whittington & Bogacz, 2017).

A.2. Inversion

Unlike standard feedforward neural networks, which operate through a strictly one-directional forward pass from inputs to outputs, PCNs perform inference via recurrent, bidirectional dynamics that minimize a global energy functional. This structure naturally enables what we refer to as *inversion*: the generation of input-level representations from fixed higher-level states (Rao & Ballard, 1999; Sun & Orchard, 2020).

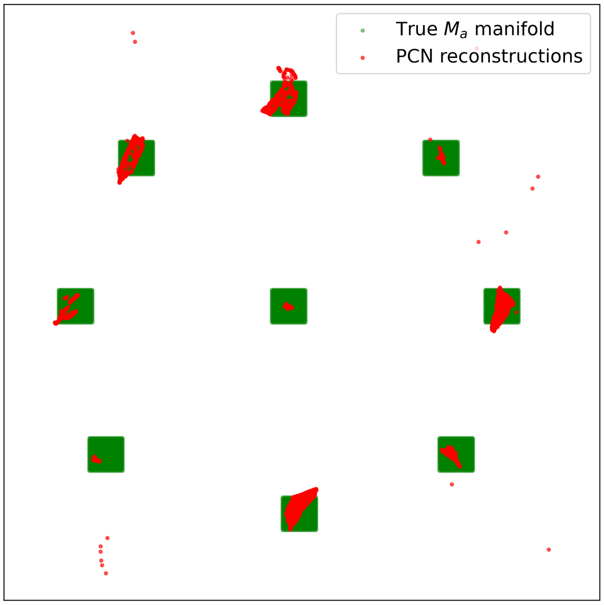
In our implementation, inversion is performed by clamping the output-layer target and then directly optimizing the input through a top-down reconstruction of the lower layers that best explains that imposed state under the learned generative parameters. Concretely, we solve an energy-minimization problem of the form

$$\hat{z} \in \operatorname*{argmin}_{z} F(z, y_{\text{target}}), \tag{A.6}$$

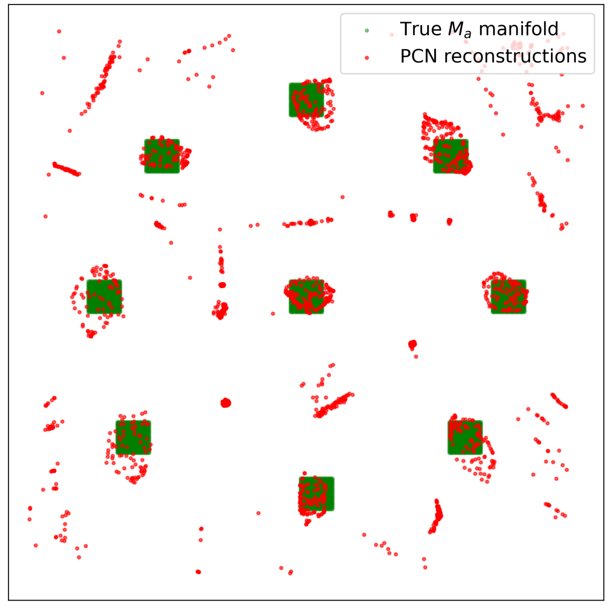
where y_{target} is a one-hot class vector and F is the variational free energy (Equation (A.3)).

To keep reconstructions bounded, we optimize this unconstrained auxiliary variable $\hat{z}$ and set $\hat{x} = \tanh(\hat{z})$ to be our "reconstruction", ensuring $\hat{x} \in (-1, 1)^d$; this matches the natural domain of our datasets. Starting from a random initialization of z, we perform a fixed number of gradient descent steps on the energy function F. This stochasticity allows the same target output to yield a distribution of plausible reconstructions rather than a single deterministic result.

We leverage this inversion mechanism in Definition 3.2 to assess the internal consistency and representational fidelity of the learned model; see Figure A.1 for a visualization.



(a) Reconstructed inputs from `30x8_leaky`



(b) Reconstructed inputs from `30x4_10x4_relu`

Figure A.1. Reconstructed inputs from two PCNs (red), overlaid on the true manifold M_a (green). Reconstructions by `30x8_leaky` (left) remain localized within components, while those by `30x4_10x4_relu` (right) are substantially more scattered and disconnected, indicating weaker approximate invertibility, aligning with the former's significantly lower MRD (0.024 vs. 0.122, respectively).

B. Model Details

Below is a summary of the models used in our experiments, including activation functions used and the number of neurons in each layer. Our implementation of Leaky ReLU weighs negative inputs by 0.2.

Name	**Activations**	**Neurons in Each Layer**
30x8	ReLU, tanh, Leaky ReLU	30, 30, 30, 30, 30, 30, 30, 30
24x8	ReLU, tanh, Leaky ReLU	24, 24, 24, 24, 24, 24, 24, 24
18x8	ReLU, tanh, Leaky ReLU	18, 18, 18, 18, 18, 18, 18, 18
30x4_24x4	ReLU, tanh, Leaky ReLU	30, 30, 30, 30, 24, 24, 24, 24
30x4_18x4	ReLU, tanh, Leaky ReLU	30, 30, 30, 30, 18, 18, 18, 18
30x4_12x4	ReLU, tanh, Leaky ReLU	30, 30, 30, 30, 12, 12, 12, 12
30x4_10x4	ReLU, tanh, Leaky ReLU	30, 30, 30, 30, 10, 10, 10, 10
30_28_26_24_22_20_18_16	ReLU, tanh, Leaky ReLU	30, 28, 26, 24, 22, 20, 18, 16
30_25_20_15_10x4	ReLU, tanh, Leaky ReLU	30, 25, 20, 15, 10, 10, 10, 10
30_15_30x6	ReLU, tanh, Leaky ReLU	30, 15, 30, 30, 30, 30, 30, 30
(30_15)x4	ReLU	30, 15, 30, 15, 30, 15, 30, 15
15x4_30x4	ReLU, tanh	15, 15, 15, 15, 30, 30, 30, 30

Table B.1. Summary of architectures trained on the synthetic dataset with their hidden layer widths and activation functions used.

Name	**Activations**	**Neurons in Each Layer**
512x8	ReLU, tanh, Leaky ReLU, Softplus	512, 512, 512, 512, 512, 512, 512, 512
256x8	ReLU, tanh, Leaky ReLU, Softplus	256, 256, 256, 256, 256, 256, 256, 256
512_32	ReLU, tanh, Leaky ReLU, Softplus	512, 384, 256, 192, 128, 96, 64, 32
512_256_128_64	ReLU, tanh, Leaky ReLU	512, 512, 256, 256, 128, 128, 64, 64
512_32_bottleneck	ReLU, tanh	512, 256, 128, 64, 32, 64, 128, 256

Table B.2. Summary of architectures trained on MNIST with their hidden layer widths and activation functions used.

C. Topology Tracking Procedure

C.1. Synthetic Data

We adopt and extend the topological analysis framework of Naitzat et al. (2020) to study how the topology of a point-cloud dataset evolves as it propagates through the hidden layers of a PCN. All models are first trained to convergence, achieving at least 99.9% test accuracy.

As in Naitzat et al. (2020), we compute and track persistent homology on only one of the two manifolds in the synthetic dataset, denoted M_a in Figure 1. Concretely, before extracting layer-wise representations, we filter the dataset to retain only points belonging to M_a (i.e., a single class) and then compute Betti numbers on this restricted point cloud. This restriction is necessary because the full dataset is a union of two manifolds; computing homology on the union would conflate the topology of manifold M_a with that of manifold M_b and with inter-manifold separations, obscuring the within-manifold simplification behavior we aim to measure.

To ensure comparability across layers with varying activation scales, we compute topology using the unweighted geodesic distance on a k-NN graph constructed over each layer's representations. By assigning unit weight to all k-NN edges, this metric normalizes distances within each latent space, reducing sensitivity to absolute scale and enabling the use of a fixed persistent homology threshold $\eta = 2.5$ across layers.

The choice of the k-NN parameter k is critical: a k too small results in a disconnected graph ($\beta_0 >$ ground truth), while a k too large induces "short-circuiting" that collapses the manifold's intrinsic structure. To determine an optimal k for the synthetic dataset, we perform a Monte Carlo cross-validation procedure:

1. Subsampling: In each of $n = 1000$ trials, we randomly sample a fraction (25%) of the synthetic manifold to simulate varying densities.
2. Iterative Search: For each subset, we iteratively increase k from 1 to 100, constructing the corresponding geodesic distance matrix.
3. Homological Matching: By computing the zeroth Betti number (β_0) at the output layer, we can identify the minimal k required to recover the ground-truth topology of manifold M_a ($\beta_0 = 9$).

The distribution converges to a mode of $k = 14$. This value consistently recovers the true connectivity of the manifold while remaining below the threshold of over-connectivity. To maintain consistency across all comparative experiments, we fix $k = 14$ and utilize a single representative 25% subset whose output-layer topology exactly matches the ground-truth manifold signature.

Persistent homology is then computed via Vietoris–Rips complexes using the `Ripser` Python package (Bauer, 2021), itself relying on foundational work from Zomorodian & Carlsson (2004) and Edelsbrunner et al. (2002). A crucial result comes from Attali et al. (2013), showing that for a large enough sample size and at a sufficiently small scale, the topology of the VR-complex of a manifold reflects the true topology of that manifold.

C.2. Real-World Data

For real-world datasets such as MNIST, the true topology of the underlying data manifold is unknown. Consequently, unlike in synthetic settings, there is no way to select or validate a k-NN construction by verifying recovery of the correct input-layer topology, motivating the use of other distance metrics.

The most immediate alternative is Euclidean distance; however, when computing persistent homology on layer-wise activations, raw Euclidean distances are not directly comparable across layers due to arbitrary expansions or contractions induced by network weights (Naitzat et al., 2020). Without normalization, changes in persistent homology may therefore reflect trivial scaling effects rather than meaningful changes in representational structure.

To address this issue, we follow Wheeler et al. (2021), who adopt a simple per-layer metric normalization procedure: For a fixed layer ℓ, let $X^{(\ell)} = \{x_1^{(\ell)}, \dots, x_n^{(\ell)}\} \subset \mathbb{R}^{n_\ell}$ denote the activations of n distinct data points at that layer. We compute the full pairwise Euclidean distance matrix and rescale it by its maximum entry, defining a normalized metric

$$d_\ell(x_i, x_j) = \frac{\|x_i^{(\ell)} - x_j^{(\ell)}\|_2}{\max_{p,q} \|x_p^{(\ell)} - x_q^{(\ell)}\|_2}. \tag{C.1}$$

This normalization ensures that the diameter of the metric space $(X^{(\ell)}, d_\ell)$ is equal to one for every layer, thereby placing all layers on a common distance scale.

To ensure that this normalization does not alter the topology recovered at any fixed layer, we note that global rescaling of a metric space induces an isomorphic persistent homology module after a corresponding rescaling of the filtration parameter (Edelsbrunner & Harer, 2010). Betti numbers at any fixed filtration threshold are therefore preserved up to this reparametrization, and comparisons across layers remain valid.

Persistent homology is otherwise calculated identically as in Section C.1. For computational efficiency, we restrict attention to homology dimensions $k \in \{0, 1\}$, which capture connected components and one-dimensional loops, respectively.

D. Topological Background and Persistent Homology

This section briefly summarizes the topological tools used throughout the paper. Our goal is not to develop new theory, but to fix notation and justify the use of Betti numbers and persistent homology as diagnostic measures of representational structure in PCNs.

D.1. Betti Numbers as Measures of Representational Complexity

Betti numbers provide coarse but informative summaries of topological structure. For a topological space X, the zeroth Betti number $\beta_0(X)$ counts connected components, while $\beta_1(X)$ counts one-dimensional holes. In the context of neural representations, these quantities serve as proxies for topological complexity: high β_0 indicates fragmented representations, while nonzero β_1 reflects loop-like structure in the data manifold.

Following prior work on the topology of deep networks, we interpret decreases in Betti numbers across layers as evidence of topological simplification, corresponding to irreversible mergers or eliminations of homological features. Because such changes cannot be undone by continuous maps, Betti-number collapse provides a natural notion of structural information loss.

D.2. Persistent Homology

Persistent homology tracks how topological features appear and disappear across a filtration parameter, rather than at a single scale. Given a filtered space $\{X_r\}_{r\in\mathbb{R}}$, persistent homology associates to each dimension d a collection of intervals $[b, d)$, known as a barcode, where each interval records the lifespan of a d-dimensional feature.

The central premise is that features with long persistence reflect meaningful structure, while short-lived features are likely due to noise. For our purposes, persistent homology provides a stable and scale-aware method for estimating Betti numbers from finite samples of high-dimensional representations.

D.3. Why Persistent Homology for Predictive Coding Networks

Predictive coding networks differ fundamentally from feed-forward architectures in that they support approximate inversion via recurrent inference. This makes them uniquely sensitive to irreversible representational collapse: once topological features are merged or destroyed, the information required for faithful reconstruction is no longer available.

Persistent homology provides a principled way to detect precisely these irreversible events. In particular, early reductions in Betti numbers indicate that topological distinctions in the input manifold have already been eliminated, constraining the space of realizable reconstructions. This directly motivates our use of Betti-based metrics to study the trade-off between abstraction and reconstruction in PCNs.

D.4. Scope and Limitations

We emphasize that persistent homology is used here as a descriptive probe, not as a complete characterization of representation geometry. Betti numbers capture only coarse topological structure and are blind to metric distortions. Nevertheless, their robustness and interpretability make them well-suited for identifying structural transitions that are invisible to purely geometric statistics.